

微控制器課程成果報告

以單板電腦搭配邏輯控制電路
實現計算機之功能

指導教授：江士標 教授

110303524 潘佳成

中華民國 110 年 1 月 15 日

目錄

一、 摘要.....	3
二、 背景.....	3
(一)資料匯流排(Data Bus).....	3
(二)位址匯流排(Address Bus).....	3
(三)位址解碼器與位址解碼 IC	4
(四)輸出埠緩衝栓鎖(OutPort Data buffer Latch).....	5
(五)三態輸入開關.....	6
(六)七節管碼.....	7
(七)16 宮格鍵盤	8
三、 硬體設計.....	8
(一)位址解碼及讀寫控制電路.....	8
(二)4X4 鍵盤經緯線配線電路	9
(三)計算機硬體配線.....	10
四、 程式控制.....	12
(一)鍵盤輪詢函式(getkeyboard).....	12
(二)七節管顯示函式(putLED).....	14
(三)運算結果顯示函式(put_result).....	16
(四)輸入數字整合函式(sum).....	18
(五)計算函式(calculate).....	20
(六)主函式(main).....	21
五、 結論.....	25
六、 參考資料.....	25

一、摘要

本實驗目的為使用 AVR128 硬體實驗板、九宮格鍵盤及七節管，以 C 語言編寫程式，製作出一個具備加減乘除功能的按鍵式計算機。應用邏輯電路控制微控制器與多個外圍設備之間的訊號選擇與傳輸。

為了實現位址解碼，使用了 74LS138 這類 3-8 線譯碼器，通過將微控制器的位址訊號解碼為具體的選擇信號，使微控制器可以依據不同的位址來控制九宮格鍵盤或七節管顯示器等外圍元件。通過 74LS374 來實現資料緩衝，暫時保存來自微控制器的資料，並根據需求將資料傳輸到選中的設備，避免訊號衝突，使多個設備可以共用同一個資料匯流排而不會發生訊號干擾。

此外，鍵盤的輸入透過對經緯線的輪詢來讀取，當按鍵被按下時，對應的數值會儲存並顯示在七節管上，七節管顯示的控制則通過位址解碼來選擇需要驅動的七節管，並由資料緩衝器將對應的數據傳輸到顯示器上，最終完成顯示。

軟體部分編寫了多個函式，包括鍵盤輪詢、七節管顯示、運算結果顯示等。整體來說，這些邏輯電路元件的整合工作確保了微控制器能夠準確、穩定地利用程式控制外圍設備，實現計算結果的顯示和連續運算的功能。

二、背景

(一)資料匯流排(Data Bus)

電腦系統結構以 CPU 為核心，並搭配記憶體及周邊電路組成。為了進行訊號傳輸，這些元件之間必須透過導線將訊號的發送端與接收端連接起來。最簡單的訊號傳遞方式是直接連線，將多組資料信號線分別連接到不同的記憶體與 I/O 裝置，從而構成整個系統。然而，這種結構雖然直觀，但其缺點在於接線過多，且每增加一個元件，就需要額外增加多條信號線。為了解決接線過多的問題，最有效的方法是共用通訊埠，也就是使用資料匯流排。透過這種方式，只需使用一組接線即可控制多個元件，大大減少了接線的複雜度。

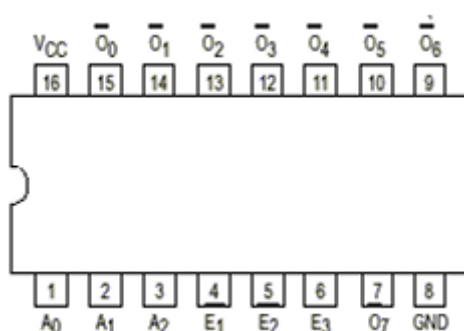
(二)位址匯流排(Address Bus)

雖然資料匯流排可以減少接線，但在同一時間內仍然只能允許一組訊號進行傳輸。因此，我們需要找到一種方法來指定當前與 CPU 溝通的元件，以解決匯流排使用權的問題。當電腦送出位址指標時，該指標指向的裝置就擁有使用資料匯流排的權限。而讀寫操作則由 CPU 透過讀寫控制線進行控制。CPU 會利用位址匯流排將位址編碼發送給所有元件，當某個元件接收到的位址編碼與其自身位址相符時，該元件便取得資料匯流排的使用權。

(三)位址解碼器與位址解碼 IC

CPU 通過位址匯流排將其指定的讀寫資料位址發送給所有相關裝置，這些裝置則會比對接收到的位址，以判斷自己是否是 CPU 所要讀寫的對象。由於位址是以二進位編碼表示，因此可用的位址總數為 2 的冪次方。如果位址線有 n 條，則可分配的位址數為 2^n 。此時，便需要位址解碼器來進行位址信號的解碼和轉換。

在本實驗中，我們使用 74LS138 作為位址解碼器。74LS138 是一個 3 對 8 的解碼器，這表示它能夠將 3 條位址線解碼為 8 條輸出，從而控制多達八個擴充的輸入/輸出埠。



(74LS138)

TRUTH TABLE

INPUTS						OUTPUTS							
E1	E2	E3	A0	A1	A2	O0	O1	O2	O3	O4	O5	O6	O7
H	X	X	X	X	X	H	H	H	H	H	H	H	H
X	H	X	X	X	X	H	H	H	H	H	H	H	H
X	X	L	X	X	X	H	H	H	H	H	H	H	H
L	L	H	L	L	L	L	H	H	H	H	H	H	H
L	L	H	H	L	L	H	L	H	H	H	H	H	H
L	L	H	L	H	L	H	H	L	H	H	H	H	H
L	L	H	H	H	L	H	H	H	L	H	H	H	H
L	L	H	L	L	H	H	H	H	H	L	H	H	H
L	L	H	H	L	H	H	H	H	H	H	L	H	H
L	L	H	L	H	H	H	H	H	H	H	H	L	H
L	L	H	H	H	H	H	H	H	H	H	H	H	L

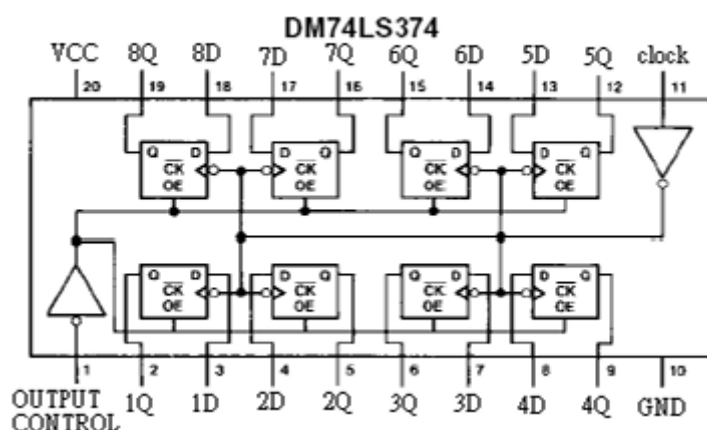
H = HIGH Voltage Level
L = LOW Voltage Level
X = Don't Care

(74LS138 真值表)

由真值表可以看到， $E1=0$ 、 $E2=0$ 、 $E3=1$ 時這顆 IC 致能，輸出才會受到輸入影響。當這顆 IC 致能時， $A0$ 、 $A1$ 、 $A2$ 三支腳共三個位元，以二進制表示 0 到 7，其所表示的值即為輸出為 L 的腳。例如 $A2=1$ ， $A1=1$ ， $A0=0$ 則 $O6$ 輸出為 L，其它的為 H。 $O0\sim O7$ 8 支腳可以拿來選擇致能被解到住址的 IC。

(四) 輸出埠緩衝栓鎖(OutputPort Data buffer Latch)

資料匯流排 (Data Bus) 是多個元件與記憶體與 CPU 之間共享的資料傳輸通道。每個元件或記憶體只有在 CPU 指定並透過位址匯流排 (Address Bus) 通知其進行讀寫操作時，才能獲得資料匯流排的使用權。然而，我們希望當元件接收到訊號後，即使在不再擁有使用權的期間內，其輸出準位仍然保持不變。如果沒有這種機制，像七段顯示器這類設備只會在接收到輸入的瞬間閃爍，而無法持續顯示預期的內容。因此，輸出埠資料緩衝鎖存器的作用便是保持元件的輸入訊號穩定，直到下一次訊號變更。



(74LS374)

DM74LS374

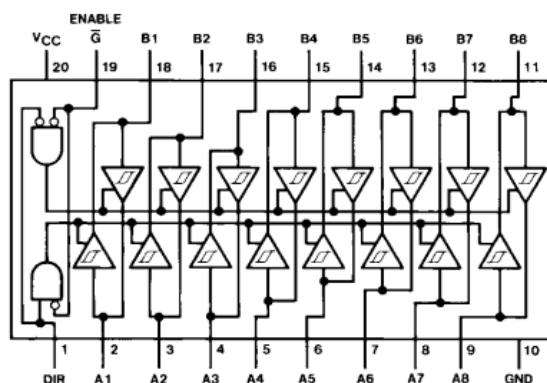
Output Control	Clock	D	Output
L	↑	H	H
L	↑	L	L
L	L	X	Q ₀
H	X	X	Z

(74LS374 真值表)

74LS374 (資料栓鎖 IC) 是一個 8-bit register 具有栓鎖資料的功能。利用 74LS374，配合解碼電路，再加上寫入控制線，即可以製作出一個平行輸出埠。由真值表我們可以看到 Output Control 控制輸出，它是反相邏輯，只有在 L 時，才允許輸出，H 時輸出的 Z 表示高阻抗，亦即與輸入是斷路浮接的。在 Output Control 允許輸出的狀態下，受到 CLOCK 的上昇緣時，才會由輸入值 D，抄一份 H/L 由 Q 輸出。當缺少上昇緣時，他會維持原始的訊號 Q₀。

(五) 三態輸入開關

三態開關是一個可以由電腦以數位訊號控制的開關，之所以被稱作三態，是因為它的輸出值，可由輸入值 H/L 的變動，加上受開關控制的影響變為 H、L、Z(斷路)共三種狀態。



(74LS245)

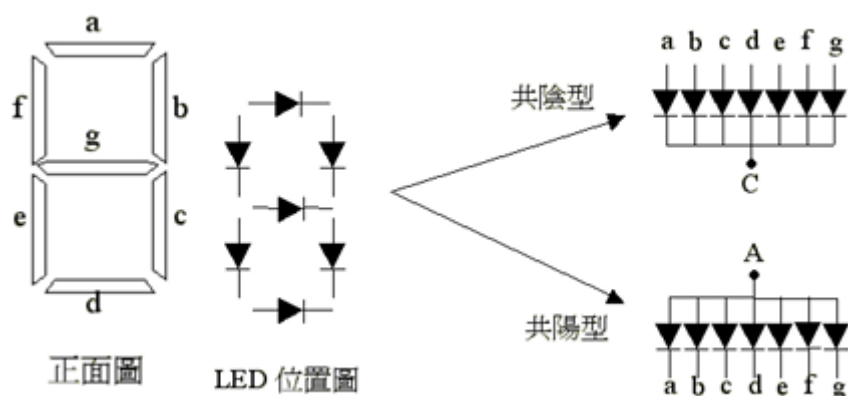
Enable G	Direction Control DIR	Operation
L	L	B Data to A Bus
L	H	A Data to B Bus
H	X	Isolation

H = HIGH Level
L = LOW Level
X = Irrelevant

(74LS245 真值表)

74LS245 為一顆三態非反向緩衝 IC。它可以一次控制八位元的開關，允許或不允許輸入端訊號通過到輸出端。他除了可以提供開關的功能外，也可以緩衝輸入到輸出，即使輸入電位比 5V 略降，也可以讓輸出電位回到 5V。由真值表可以看出 Enable 只有在 L 時 IC 才會致能，DIR 則是控制訊號傳輸方向。

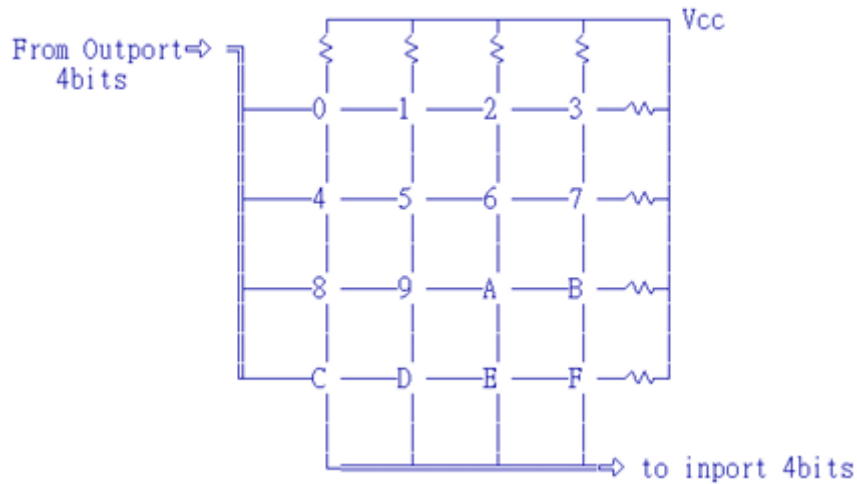
(六) 七節管碼



七節管是由七個排成 8 字型圖型的 LED 組。這七個 LED 的亮滅組合，可以分別表由 0 到 9 的不同圖型。七節管碼則是由 a~g 的排列所決定的，不同的接線方式所對應出的七節管碼也都不相同。本實驗所使用之七節管為共陽型，意即當輸出為 L 時 LED 亮，下表為本實驗的七節管碼。

portD	6	5	4	3	2	1	0		
	g	f	e	d	c	b	a	2 進制	10 進制
0	1	0	0	0	0	0	0	0b0100 0000	64
1	1	1	1	1	0	0	1	0b0111 1001	121
2	0	1	0	0	1	0	0	0b0010 0100	36
3	0	1	1	0	0	0	0	0b0011 0000	48
4	0	0	1	1	0	0	1	0b0001 1001	25
5	0	0	1	0	0	1	0	0b0001 0010	18
6	0	0	0	0	0	1	0	0b0000 0010	2
7	1	0	1	1	0	0	0	0b0101 1000	88
8	0	0	0	0	0	0	0	0b0000 0000	0
9	0	0	1	0	0	0	0	0b0001 0000	16
-	0	1	1	1	1	1	1	0b0111 1111	63

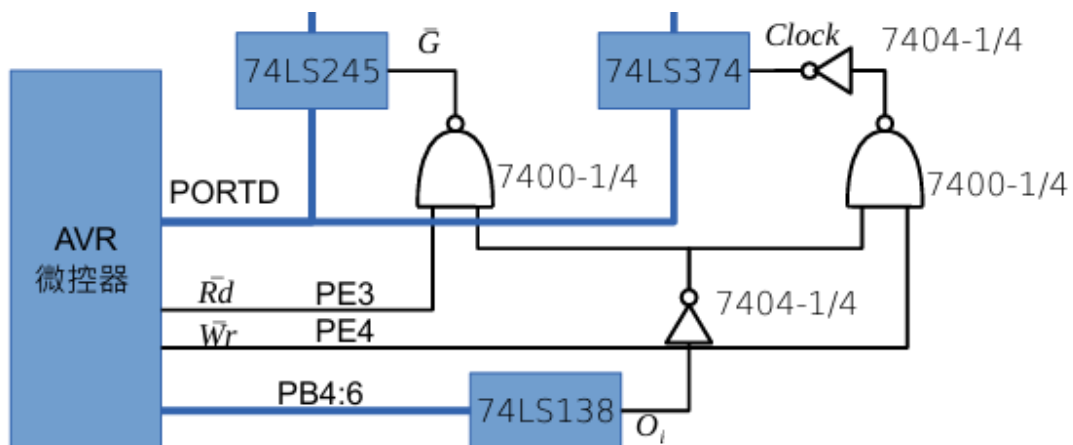
(七) 16 宮格鍵盤



經緯線總共有 16 個交點，分別由一個按鍵控制是否接通。假設如圖上所示，四條經緯線的另一端各自以限流電阻接至電源來做 PULL UP，所有經緯線一般狀況下均為高電位。若某一條緯線的一端被輸出埠控制拉到低電位，且被按下按鍵將這條緯線導通特定的經線，則這條特定經線將會被拉均至低電位，其它的經線不變。因此想偵測某一按鍵是否被按下，只要對其所跨的緯線輸出低電位，同時讀取其所跨越的經線是否為低電位，即可知道這對經緯線交叉點上的按鍵是否被按。

三、 硬體設計

(一) 位址解碼及讀寫控制電路



以 PORTE3：4 分別作為讀寫控制，來決定目前是否要將訊號寫入或寫出元件，由於只使用 PORTD 作為資料匯流排，所以使用 PORTB4：6 作為位址匯流排，用

以控制資料匯流排的使用對象。而這兩者需同時成立，才可以將訊號寫入或寫出目標元件。因 74LS138（位址解碼 IC）解碼出的位址訊號為低電平，所以先利用 NOT GATE 7404 將位址訊號改為高電平。

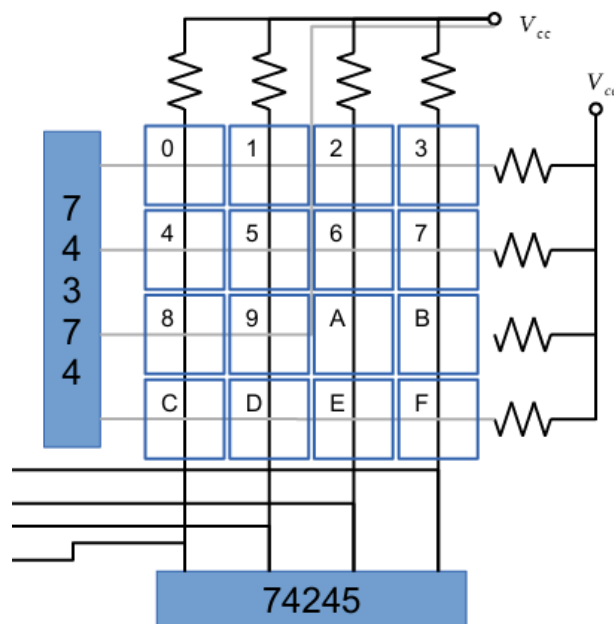
輸入控制：

使用三態開關(74LS245)來控制輸入訊號是否能進入資料匯流排，此 IC 只有在 ENABLE 為 L 時才會致能，所以使用 NAND GATE 7400 進行邏輯判斷，當輸入控制訊號為 H 且接收到對應的位址訊號時，開關導通。

輸出控制：

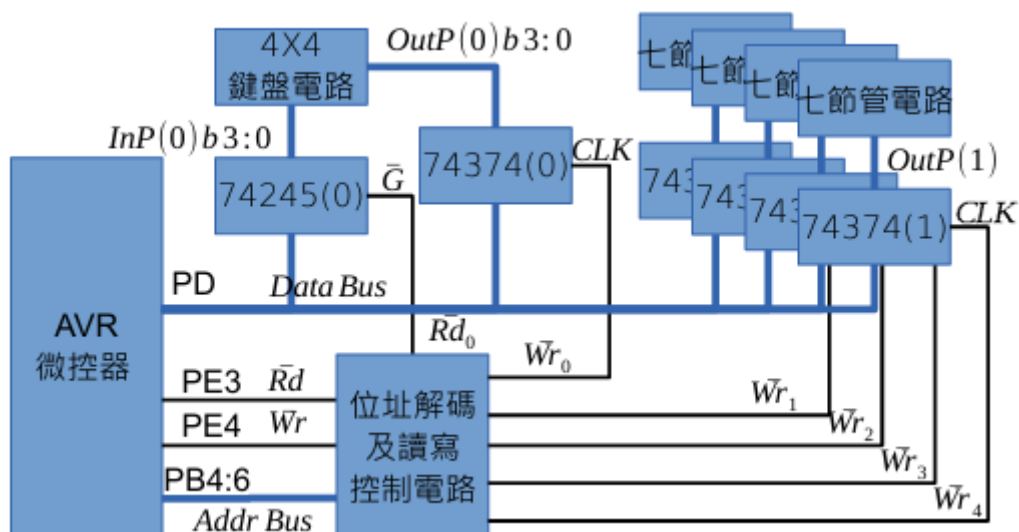
為了節省邏輯 IC 的使用，於是與輸入控制共用 7400，將輸出控制訊號(PE4)及位址訊號進行 NAND 邏輯判斷後再經過 NOT，也就是將兩訊號進行 AND 判斷，若當前輸出控制訊號為 H，則解碼訊號所對應到的元件會產生一個上生緣，使資料匯流排訊號透過輸出埠緩衝栓鎖(74LS374)複製一份進入目標元件。

(二) 4X4 鍵盤經緯線配線電路

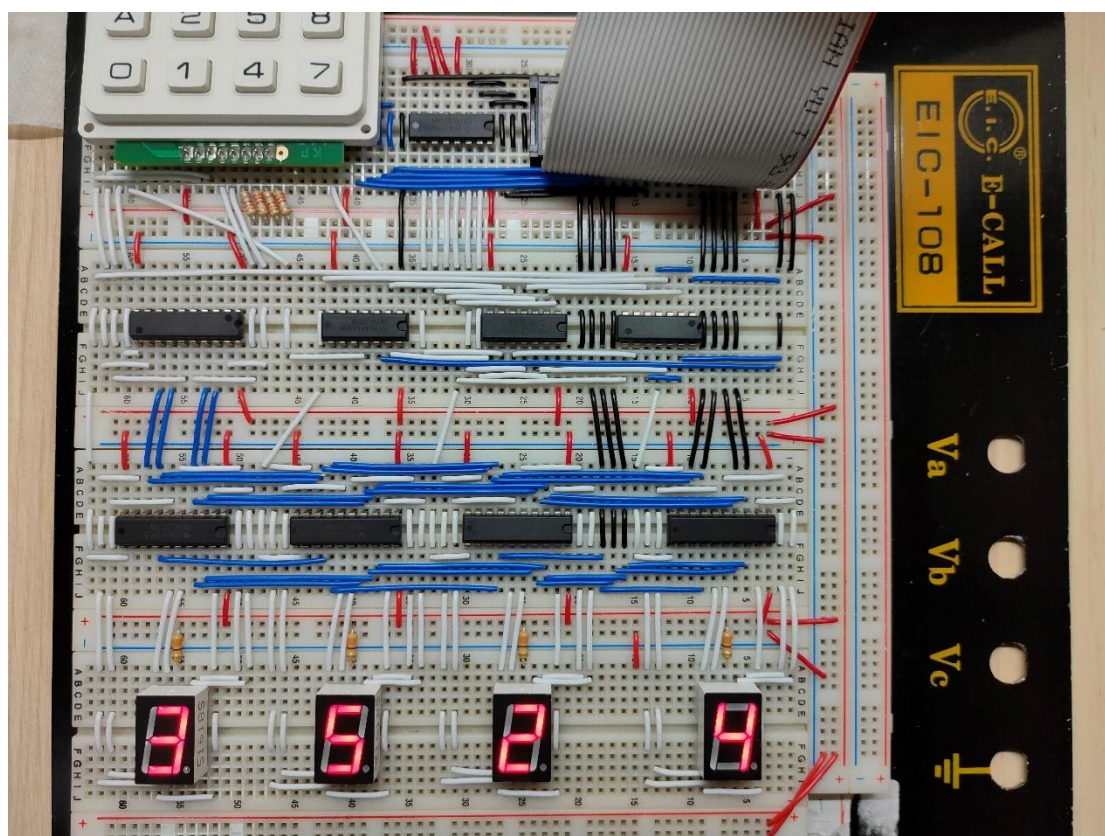


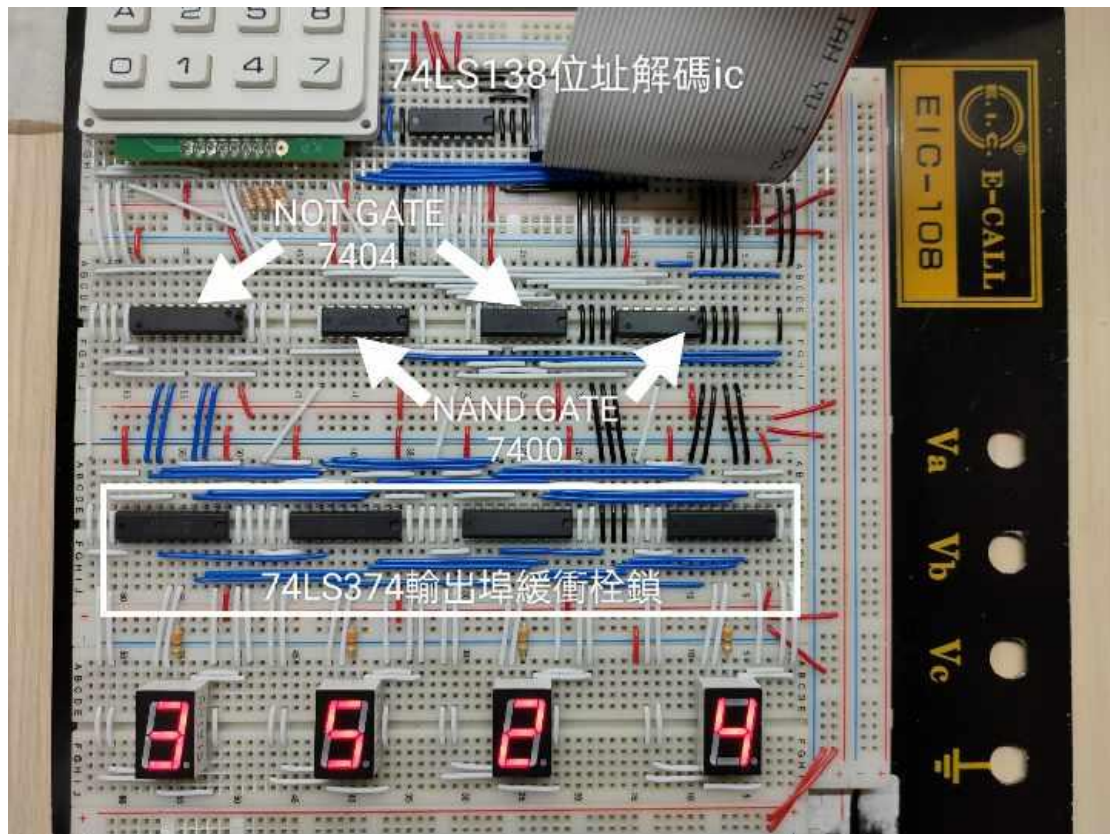
利用上述位址解碼及讀寫控制電路，先透過輸出埠緩衝栓鎖(74LS374)對鍵盤緯線輸出低電位，再藉由控制三態開關(74LS245)來讀取經線訊號。搭配程式控制進行案件偵測

(三) 計算機硬體配線



將位址解碼及讀寫控制電路所解出的六條(共八條, 74LS138 為 3 對 8 解碼器)位址訊號線分別連接至鍵盤及四顆七節管。其中鍵盤占了兩條, 分別控制輸入及輸出; 剩下四條控制輸出至各顆七節管。

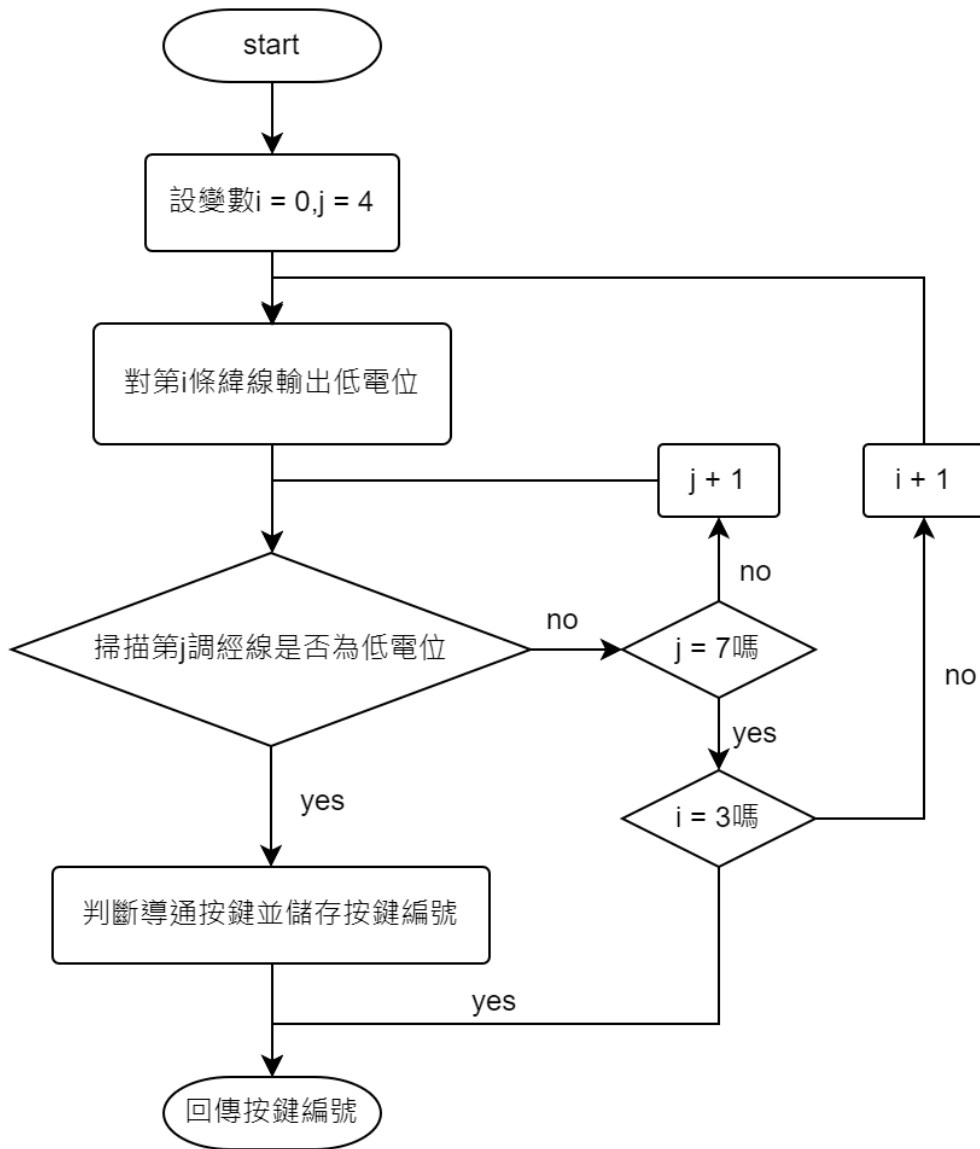




(實際線路)

四、程式控制

(一)鍵盤輪詢函式(getkeyboard)



(程式流程圖)

由於鍵盤上除了數字外並沒有 $+-\times\div$ 等符號，而是英文字母，於是先將鍵盤上個按鍵編號如下圖。並且設定英文字母 A~F 分別為 $=$ 、clear、 $+$ 、 $-$ 、 $\times$ 、 $\div$ 。



12 (÷)	13 (×)	14 (-)	15 (+)
11 (clear)	3	6	9
10 (=)	2	5	8
0	1	4	7

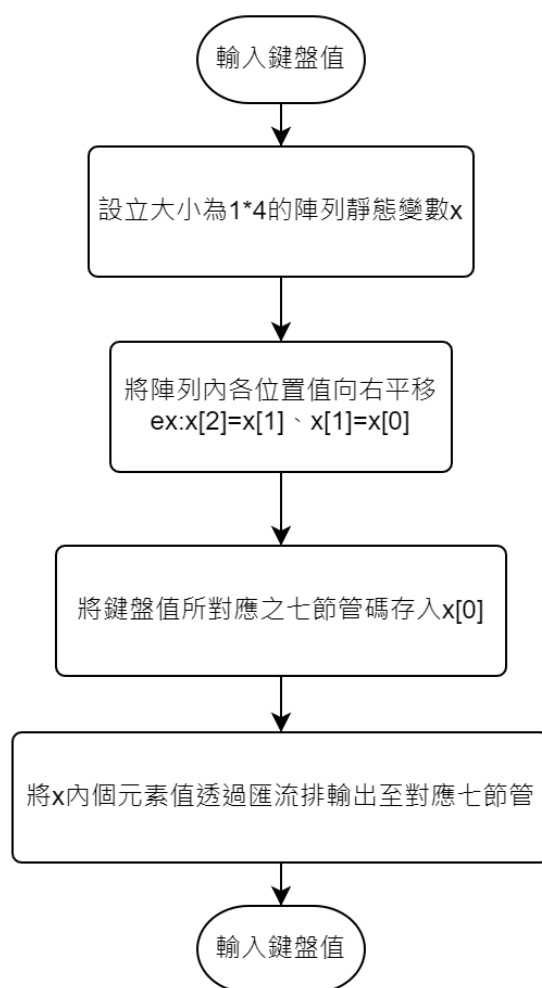
```
int getkeyboard(){
    static int keyboard[4][4]={12,13,14,15},{11,3,6,9},{10,2,5,8},{0,1,4,7}};
    uint8_t n,j;
    uint8_t get;
    int m=20;
    REGFPT(&DDRB,0xFF,0,0xFF);
    REGFPT(&DDRE,0x18,0,0b00011000);
    REGFPT(&DDRD,0xFF,0,0x0F);
    for(n=0;n<4;n++){
        //輪流將PORTD 0:3腳位設為L，四個bit分別對應至鍵盤四條緯線
        REGFPT(&PORTE,0xFF,0,0b00000000);
        REGFPT(&PORTD,0x0F,0,~(1<<n));
        _delay_ms(1);
        //透過位址匯流排將暫存器資訊輸出至鍵盤
        REGFPT(&PORTB,0xFF,4,5);
        REGFPT(&PORTE,0xFF,0,0b00010000);
        _delay_ms(1);
        for(j=4;j<=7;j++){
            //透過位址匯流排開啟鍵盤輸入權限
            REGFPT(&PORTB,0xFF,4,4);
            REGFPT(&PORTE,0xFF,0,0b00010000);
            _delay_ms(1);
            //輪流讀取PORTD 4:7腳位，並將訊號存入"get"，四個bit分別對應至鍵盤四條經線
            REGFGT(&PIND,1<<j,j,&get);
            REGFPT(&PORTE,0xFF,0,0b00000000);
            //當get為0時，比對目前經線編號即可得知哪一個鍵被按下
            if(get==0){
                m = keyboard[n][j-4];
                DDRB=0xff;
                REGFPT(&PORTB,0xFF,4,4);
                REGFPT(&PORTE,0xFF,0,0b00010000);
                //直到按鍵鬆開才離開函式，避免重複讀取
                do {
                    _delay_ms(1);
                    REGFGT(&PIND, 0xf0, 0, &get);
                }while(get!=0xf0);
                REGFPT(&PORTE,0xFF,0,0b00000000);
                break;
            }
        }
    }
}
```

(程式碼)

(二) 七節管顯示函式(putLED)

為了還原一般計算機顯示數字情形，需要讓鍵入數字逐步顯示於七節管上，而不是一次顯示。舉例來說，當輸入 3524 時，七節管顯示流程應如下表：

			3
		3	5
	3	5	2
3	5	2	4



(流程圖)

```

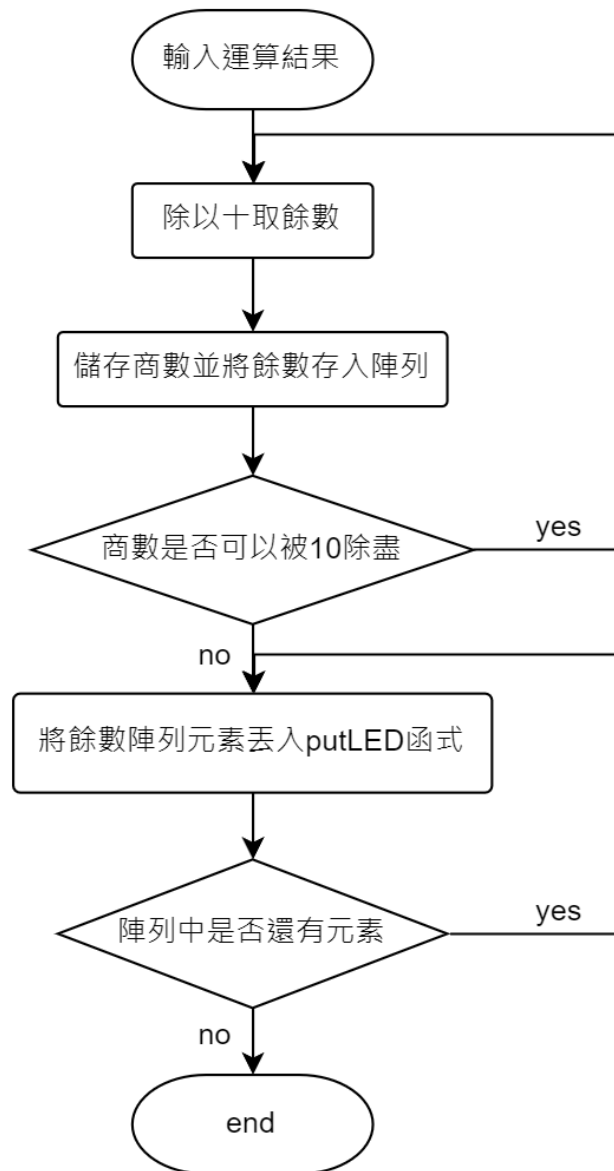
void putLED(int y){
    static int x[4]={127,127,127,127};
    int i;
    REGFPT(&DDRB,255,0,0b01110000);
    REGFPT(&PORTE,0xFF,0,0b00000000);
    REGFPT(&DDRE,0xFF,0,0b00011000);
    REGFPT(&DDRD,255,0,255);
    for(i=3;i>=1;i--){
        x[i]=x[i-1];
    }
    x[0] = SevenSegTable[y];
    for(i=3;i>=0;i--){
        REGFPT(&PORTE,0xFF,0,0b00000000);
        REGFPT(&PORTD,0xFF,0,x[i]);
        _delay_ms(1);
        REGFPT(&PORTB,0xFF,4,3-i);
        REGFPT(&PORTE,0xFF,0,0b00010000);
        _delay_ms(1);
        REGFPT(&PORTB,0xFF,4,7);
        REGFPT(&PORTE,0xFF,0,0b00000000);
        _delay_ms(3);
    }
}

```

(程式碼)

(三) 運算結果顯示函式(put_result)

由於每個七節管都是獨立的，需要個別控制。假設運結果為 3524，則需先將其拆分為 3、5、2、4，再分別送至四個七節管顯示。



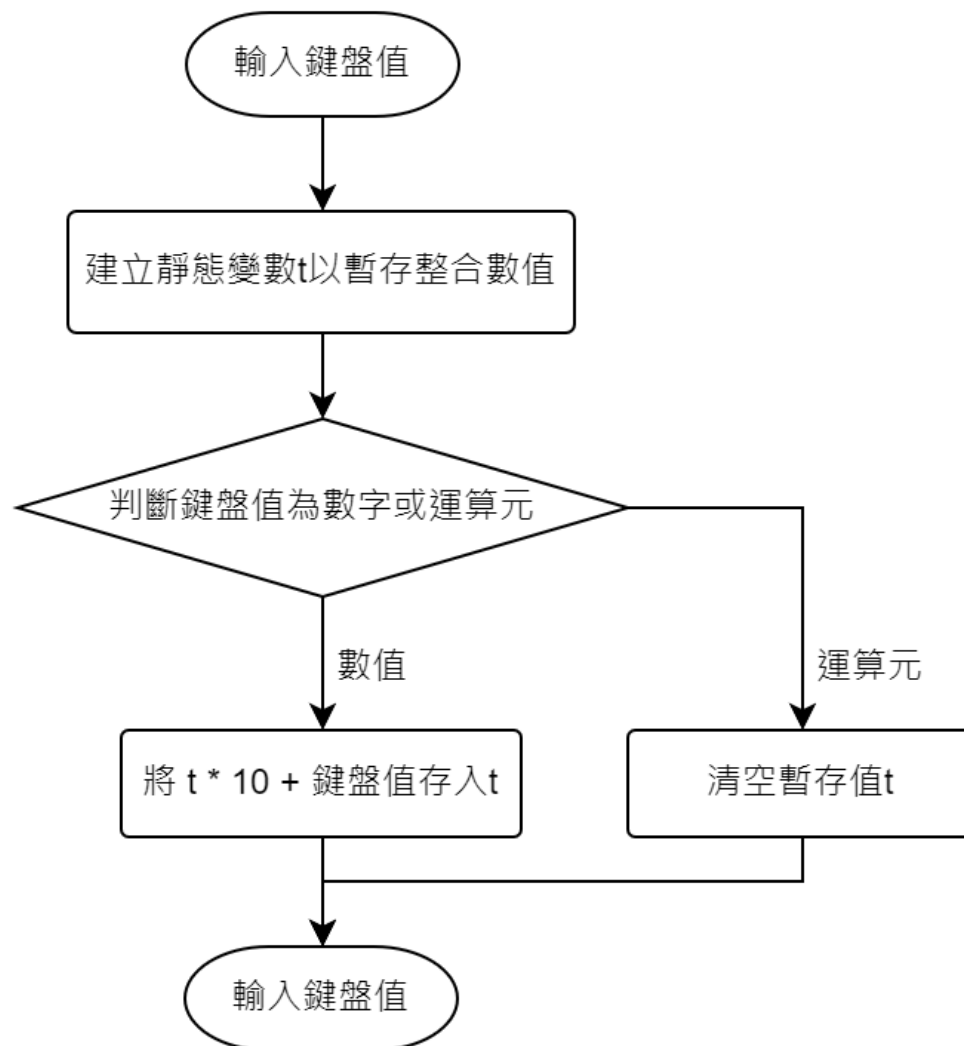
(流程圖)

```
void put_result(int value2){  
    long int res,put[4];  
    int i;  
    int cnt=0;  
    res=value2;  
    while(res>=10){  
        res=res/10;  
        cnt++;  
    }  
    for(i=3;i>=3-cnt;i--){  
        put[i]=value2%10;  
        value2=value2/10;  
    }  
    for(i=3-cnt;i<=3;i++){  
        putLED(put[i]);  
    }  
}
```

(程式碼)

(四)輸入數字整合函式(sum)

與數字顯示原理相同，由於數值是分開輸入，例如 3、5、2、4，需透過程式將其整合成 3524。

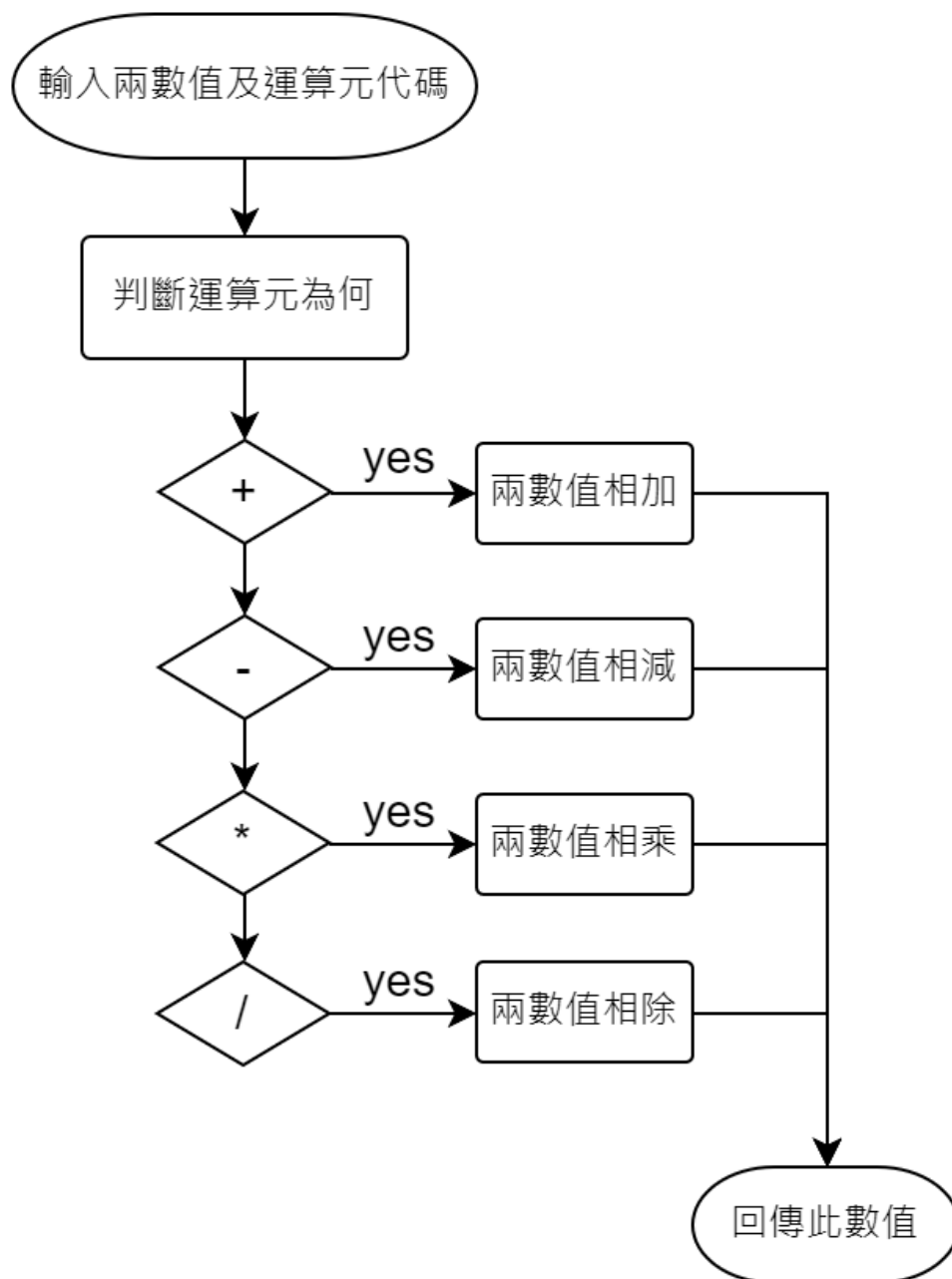


(流程圖)

```
int sum(int x){
    static long int t=0;
    long int s;
    if(x<=9){
        t=t*10+x;
        s=t;
    }
    else{
        s=t;
        t=0;
    }
    return s;
}
```

(程式碼)

(五)計算函式(calculate)



(流程圖)

```

void calculate(long int *v1,long int v2,int sign){
    switch(sign){
        case 12:
            *v1=(*v1)/(v2);
            break;
        case 13:
            *v1=(*v1)*(v2);
            break;
        case 14:
            *v1=(*v1)-(v2);
            break;
        case 15:
            *v1=(*v1)+(v2);
            break;
    }
}

```

(程式碼)

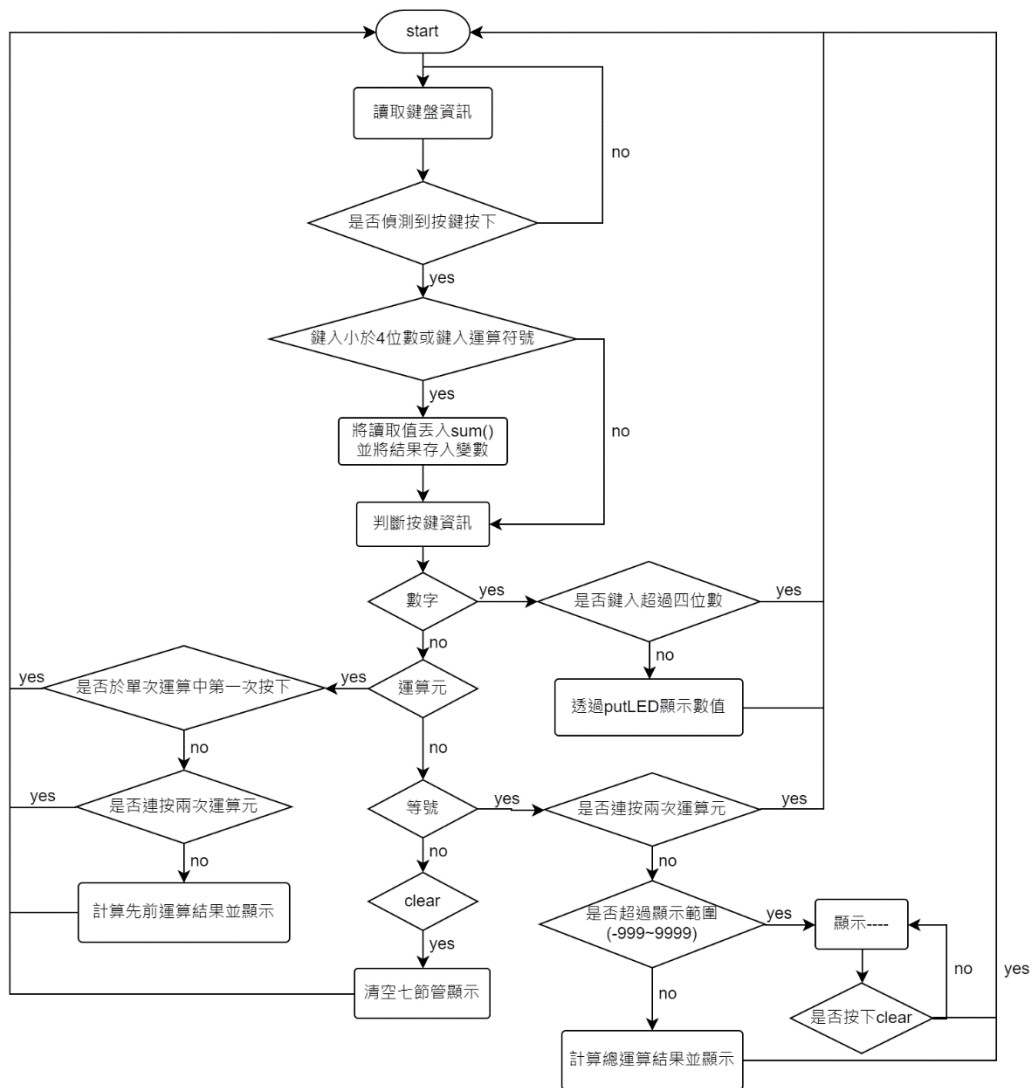
(六)主函式(main)

本計算機具備加減乘除之功能，其中英文字母 A~F 分別為=、clear、+、-、×、÷。可進行連續運算但只會依次執行，且會於按下下一個運算元時顯示先前運算結果。ex：30 + 40 * 5 = 350，以下為各按鍵按下時顯示流程表：

按下按鍵

3		3
0	3	0
+	3	0
4		4
0	4	0
*	7	0
5		5
=	3 5	0

若除數為零或超過顯示範圍(-999~9999)，則顯示----，直到按下 clear。



(流程圖)

```

while(1){
    do{
        get=getkeyboard();
    }while(get==20||!(get==0&&num_count==0&&count==0));
    if(value1==0&&sign==15){//除數等於零時顯示----
        for(i=0;i<4;i++){
            putLED(17);
        }
        do{
            get=getkeyboard();
        }while(get!=11);
    }
    if(num_count<4||get>9){
        /*若鍵入小於4位數或鍵入運算符號，則把鍵入數字加總。
        例如第一次輸入2，第二次輸入4，則將24存入tmp*/
        tmp=sum(get);
    }
    switch(get){
        case 0 ... 9:
            if(equal_sign_flag==1){//當上次為等號，清空所有計數器、旗標及數值
                initial();
                count=0;
                sign=0;
                value2=0;
                sign_flag=0;
                equal_sign_flag=0;
            }
            value1=tmp;//保留運算符號前的數值
            if(sign_flag==1){//若有按下運算符號，則清空七節管
                initial();
            }
            if(num_count<4){//無法鍵入超過4位數
                putLED(get);
            }
            sign_flag=0;
            num_count++;
            break;
        case 12 ... 15:
            if(count==0){
                value2=value1;
            }
    }
}

```

```

else{
    if(sign_flag==0){//當上次為數值才執行
        calculate(&value2,value1,sign);
        initial();
        put_result(value2);
    }
}
num_count=0;
sign_flag=1;
count++;
sign=get;//保留上次壓的符號
break;
case 10:
    if(sign_flag==0){//當上次為數值才執行
        calculate(&value2,value1,sign);
    }
    if(value2>9999){//超過顯示範圍
        for(i=0;i<4;i++){
            putLED(17);//顯示----
        }
        do{
            get=getkeyboard();
        }while(get!=11);
    }
    if(value2>=0){
        initial();
        put_result(value2);
    }
    if(value2<0){
        if(value2<-999){//超過顯示範圍
            for(i=0;i<4;i++){
                putLED(17);//顯示----
            }
            do{
                get=getkeyboard();
            }while(get!=11);
        }
        else{
            tmp=-value2;
            initial();
            putLED(17);
            put_result(tmp);
        }
    }
}

```

```

}
if(get==11){
    initial();
    led_init();
    count=0;
    sign=0;
    value2=0;
    sign_flag=0;
    equal_sign_flag=0;
    num_count=0;
}
}

```

(程式碼)

五、 結論

本實驗成功實現了使用 AVR128 硬體實驗板及 C 語言程式控制來製作按鍵式計算機的目標，該計算機具備一個十六宮格鍵盤，以供使用者輸入基本數字及運算元，並利用四顆七節管來做為運算顯示。

透過實驗，我們學習了資料匯流排、位址匯流排和輸出埠緩衝栓鎖的應用，成功以單一暫存器進行鍵盤資料讀取及七節管燈號控制。程式部分實現了鍵盤輪詢、七節管顯示和數值運算等功能，使整體硬體和軟體的操作能夠相互配合。

在實驗過程中，我們也發現硬體接線和程式邏輯的複雜性。其中最有挑戰的即為硬體設計的部分，這不僅考驗對各元件的理解，還要考慮到如何在有限的空間，有條理地排列各元件，並且有效的搭建連接彼此的訊號線。

有鑑於目前鍵盤大小及七節管數量，本實驗計算機僅能做基本運算，且顯示位數僅有四碼。未來將可針對這部分進行優化，如增加按鍵以進行更多運算功能、增加七節管數量以顯示更多位數，進行更複雜運算。

六、 參考資料

1. 74LS138 datasheet

<https://www.electroschematics.com/74ls138-datasheet/>

2. 74LS374 datasheet

<https://reurl.cc/N1Dd9Q>

3. 74LS245 datasheet

<https://www.sycelectronica.com.ar/semiconductores/74LS245.pdf>